

ASSIGNMENT-7.1

Name: B.Chandu

HT. No: 2303A51737

Batch: 08

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():  
    print "Hello, AI Debugging Lab!"  
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

```
1  #Bug Explanation: In Python 3, print is a function  
2  # Task 1: Syntax Error Fix  
3  def greet():  
4      # FIX: Added parentheses because print is a function in Python 3  
5      return "Hello, AI Debugging Lab!"  
6  # Instead of printing directly, we return the string for testing  
7  result = greet()  
8  print(result)  
9  # ✅ Assert Test Cases  
10 assert greet() == "Hello, AI Debugging Lab!"  
11 assert isinstance(greet(), str)  
12 assert "AI" in greet()  
13 print("Task 1 Passed ✅")
```

```
Hello, AI Debugging Lab!  
Task 1 Passed ✅
```

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):  
    if n = 10:  
        return "Ten"  
    else:
```

```
return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output :

```
Bug Explanation:

= is assignment operator == is comparison operator

Using if n = 10: causes a SyntaxError because assignment is not allowed inside condition.
```

```
2] 1 # Task 2: Incorrect Condition Fix
    2 def check_number(n):
    3     # FIX: Changed = to ==
    4     if n == 10:
    5         return "Ten"
    6     else:
    7         return "Not Ten"
    8
    9 # ✅ Assert Test Cases
   10 assert check_number(10) == "Ten"
   11 assert check_number(5) == "Not Ten"
   12 assert check_number(0) == "Not Ten"
   13
   14 print("Task 2 Passed ✅")
   15
```

Task 2 Passed ✅

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):
    with open(filename, 'r') as f:
        return f.read()
    print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output:

```

1  #Bug Explanation: If file does not exist → FileNotFoundError Program crashes
   without error handling.
2  # Task 3: Safe File Handling
3  def read_file(filename):
4      try:
5          with open(filename, 'r') as f:
6              return f.read()
7      except FileNotFoundError:
8          return "Error: File not found."
9      except Exception as e:
10         return f"Error: {str(e)}"
11
12  # --- Creating a test file ---
13  with open("testfile.txt", "w") as f:
14      f.write("AI Debugging Success")
15  # ✅ Test Cases
16  assert read_file("testfile.txt") == "AI Debugging Success"
17  assert read_file("nonexistent.txt") == "Error: File not found."
18  assert "Error" in read_file("/invalid/path/file.txt")
19  print("Task 3 Passed ✅")

```

Task 3 Passed ✅

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

class Car:

def start(self):

return "Car started"

my_car = Car()

print(my_car.drive()) # drive() is not defined

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output:

```

1  #Bug Explanation: my_car.drive() causes: AttributeError: 'Car' object has no
   attribute 'drive'
2  # Task 4: Fix Undefined Method
3  class Car:
4      def start(self):
5          return "Car started"
6      # FIX: Added missing method
7      def drive(self):
8          return "Car is driving"
9  my_car = Car()
10 print(my_car.drive())
11 # ✅ Assert Test Cases
12 assert my_car.start() == "Car started"
13 assert my_car.drive() == "Car is driving"
14 assert isinstance(my_car.drive(), str)
15
16 print("Task 4 Passed ✅")

```

```

Car is driving
Task 4 Passed ✅

```

Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```

def add_five(value):
    return value + 5
print(add_five("10"))

```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output:

```
1 #Bug Explanation: Causes: TypeError: can only concatenate str (not "int") to str
2 #Task5
3 # Solution 1: Convert to integer
4 def add_five_cast(value):
5     return int(value) + 5
6 # ✅ Assert Test Cases
7 assert add_five_cast("10") == 15
8 assert add_five_cast(5) == 10
9 assert add_five_cast("0") == 5
10 print("Task 5 Solution 1 Passed ✅")
11 # Solution 2: Convert number to string for concatenation
12 def add_five_string(value):
13     return str(value) + "5"
14 # ✅ Assert Test Cases
15 assert add_five_string("10") == "105"
16 assert add_five_string(10) == "105"
17 assert isinstance(add_five_string(3), str)
18 print("Task 5 Solution 2 Passed ✅")
```

Task 5 Solution 1 Passed ✅

Task 5 Solution 2 Passed ✅